

The Theory of Automata & Formal Languages

AI412

By

Dr Ahmed Zakzouk

Faculty of Artificial Intelligence

Course Overview:

- Builds a rigorous foundation in formal languages, grammars, and abstract machines.
- Covers the full Chomsky hierarchy, from regular to recursively enumerable languages.
- Develops both constructive skills (designing automata and grammars) and proof skills (pumping lemmas, reductions).
- Introduces the theoretical limits of computation through decidability and undecidability.
- Concludes with an introduction to computational complexity and the P vs. NP question.
- Provides the theoretical foundation for compiler design, parsing, and algorithm analysis.

Unit 1: Introduction to Formal Languages and Automata Theory:

- This unit introduces the alphabet-string-language hierarchy and the Chomsky hierarchy that organizes the rest of the course.
- **Historical Note:** Automata theory traces back to 1930s work on computability by Turing, Church, and Gödel, and to 1950s work by Chomsky on formal grammars for linguistics; the two threads were unified into the modern theory of computation by the 1960s.
- Computer science is inherently a practical field; practitioners often prefer addressing tangible, useful problems over engaging in purely theoretical speculation.
- This is particularly true for computer science students, whose primary interest lies in working on complex, real-world applications; they generally attach little importance to theoretical issues unless they contribute to effective solutions.

- This perspective is logical and appropriate, given that without practical applications, there would be little interest in computers. However, this practical orientation raises a valid question: "Why, then, do we study the theoretical aspects? “.

- The first answer is that theory provides concepts and principles that help us understand the general nature of the discipline. Computer science encompasses a wide spectrum of specialized topics, ranging from machine design (hardware) to programming.

- Furthermore, the practical use of computers involves a vast array of intricate details that must be mastered to ensure successful application, making computer science an exceptionally broad and diverse field. Despite this diversity, fundamental principles lie at the core of the discipline. To study these principles, we construct abstract models of computers and computational processes; these models capture the essential features shared by hardware and software and serve as a foundation for many of the complex, specialized structures encountered in computer systems.

- The second reason is that the ideas we will discuss have significant, immediate practical applications; digital design, programming languages, and compilers are prime examples, though there are many others.

- The concepts studied here form a unifying thread running through vast areas of computer science, ranging from operating systems to pattern recognition.

- The third reason is that this subject offers an enjoyable and intellectually stimulating activity; it presents numerous challenging, puzzle-like problems. This is problem-solving in its purest form.

- Mathematical Preliminaries and Notation:
- **Sets:** A set is a collection of elements, without any structure other than membership.

$$S = \{0, 1, 2\}$$

The usual set operations arc union (U), intersection (∩), and difference (-), defined as:

$$S_1 \cup S_2 = \{x : x \in S_1 \text{ or } x \in S_2\},$$
$$S_1 \cap S_2 = \{x : x \in S_1 \text{ and } x \in S_2\},$$
$$S_1 - S_2 = \{x : x \in S_1 \text{ and } x \notin S_2\}.$$

The complement of a set S, denoted by S^{c}, consists of all elements not in S, the universal set U of all possible elements. If U is specified, then:

$$\overline{S} = \{x : x \in U, x \notin S\}$$

Example 1.1 If S is the set $\{a, b, c\}$, then its powerset is

$$2^S = \{\varnothing, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}.$$

Here $|S| = 3$ and $\left|2^S\right| = 8$. This is an instance of a general result; if S is finite, then

$$\left|2^S\right| = 2^{|S|}.$$

the Cartesian product of two sets:

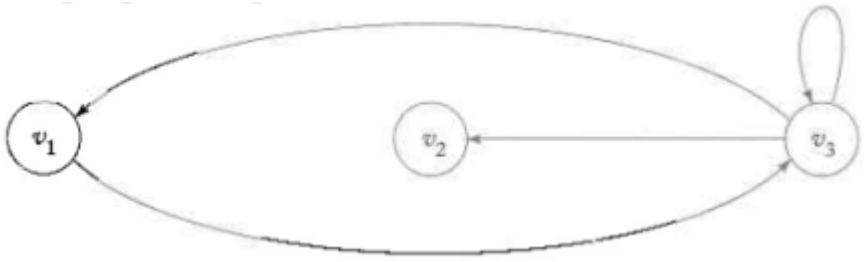
$$S = S_1 \times S_2 = \{(x, y) : x \in S_1, y \in S_2\}$$

Example 1.2 Let $S_1 = \{2, 4\}$ and $S_2 = \{2, 3, 5, 6\}$. Then

$$S_1 \times S_2 = \{(2, 2), (2, 3), (2, 5), (2, 6), (4, 2), (4, 3), (4, 5), (4, 6)\}$$

Graphs can be easily represented using charts where vertices are represented by circles, and edges by lines with arrows pointing to the vertices. Figure 1.1 shows a graph with vertices {v_1, v_2, v_3} and edges {(v_1, v_3), (v_3, v_1), (v_3, v_2), (v_3, v_3)}

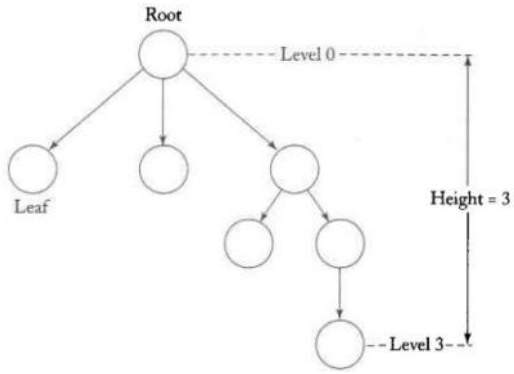
Figure 1.1



- Trees are a particular type of graph. A tree is a directed graph that has no cycles, and that has one distinct vertex, called the root, if there is an edge from v_i to v_j, then v_i is said to be the parent of v_j, and v_j the child of v_i.
- **The level** associated with each vertex is the number of edges in the path from the root to the vertex.

- **The height** of the tree is the largest level number of any vertex. These terms are illustrated in Figure 1.2.

Figure 1.2



- At this stage, we have only a general understanding of what automata are and how they can be represented by a graph. To proceed, we must be more precise, provide formal definitions, and develop rigorous results.

- We begin with "finite accepters/Automaton", This type of automaton is characterized by the absence of auxiliary storage memory. Since the input file cannot be rewritten, the finite automaton's ability to" remember" information during the computation process is extremely limited.

- we will introduce simple yet crucial definitions of terms that we will use throughout the course on automata theory and formal languages.

An alphabet $\sum$: is a non-empty finite set of symbols (or letters).

The power of that alphabet: The number of elements in an alphabet.

String: is a finite sequence of symbols chosen from some alphabet ∑. The terms string and word is equivalent.

Length of a string: Let x be a string over an alphabet Σ. For $a \in \Sigma$, the number of occurrences of a in x shall be denoted by $|x|a$. The length of a string x denoted by $|x|$ is defined as:

$$|x| = \sum_{a\in\Sigma}|x|_a$$

It is easy to see that the length of a string is obtained by counting the number of symbols in the string.

Empty string: The string of zero length is called the "empty string". This is denoted by ε. The empty string plays the role of 0 in a number system.

The Kleene star $\sum$ *: denotes the set of all possible strings over Σ, including the empty string ε

A language: is simply any set of strings over an alphabet, finite or infinite.

Examble1: Let Σ = $\{a, b, c\}$ be an alphabet. Then a, cb, aa, cba, bac, $bbaaccc$ are some strings over the alphabet Σ. But $abcd$ is not a word over the alphabet Σ, because it contains a symbol d that is not part of the alphabet $\sum$.

Examble2: Let Σ = {0,1} be a binary alphabet. Then the following are some binary strings over the alphabet Σ: 101 and 000 (strings of length 3, by other words |101| =|000| = 3), 1101 (a string of length 4), 00110 (a string of length 5), 000001 (a string of length 6). But 120 is not a string over the alphabet $\sum$, because it contains a symbol 2 that is not part of the alphabet.

Examble3: A word α = 11100 has been constructed over a binary alphabet $\sum$ = {0, 1}.

Let's determine its length: $|\alpha|$ = 5. The expression $|\alpha|_0$=2 means that the word contains 2 zeros, and the expression $|\alpha|_1$=3 means that the word contains 3 ones. The string β = 11 is another string over Σ. It is clear that $|\beta|$ = 2, $|\beta|_0$=0, $|\beta|_1$=2.

- **<u>Operations over words:</u>**

1) Reverse String: if $\omega = w_1w_2 \dots w_n$ where $wi \in \Sigma$, $1 \le i \le n$, the reverse of ω is $\omega^r = w_n\, w_{n-1} \dots w_1$. The word ω^r is the reversal of the word ω.

2) Concatenation: let $x = a_1a_2 \dots a_n$ and $y = b_1b_2 \dots b_m$ are strings, the concatenation of x and y denoted by xy is a string $a_1a_2 \dots a_nb_1b_2 \dots b_m$. It's obviously, that: $|xy| = |x| + |y|$.

Operations on Strings $\alpha, \beta, \gamma \in \Sigma^*$

Let $\alpha = a_1 \cdots a_n, \beta = b_1 \cdots b_m, \forall\, i,j : a_i, b_j \in \Sigma$

Length:	$\lvert\alpha\rvert = n, \lvert\beta\rvert = m$
Number of occurrences of q in α:	$\lvert\alpha\rvert_q, q \in \Sigma$
Concatenation:	$\alpha\beta = a_1 \cdots a_n b_1 \cdots b_m$
	$\beta\alpha = b_1 \cdots b_m a_1 \cdots a_n$
Superscript:	$w^0 = \varepsilon, w^1 = w, w^2 = ww, \ldots,$
	$w^n = ww^{n-1}$
Reverse String:	If $\alpha = a_1 a_2 \cdots a_n, a_i \in \Sigma,$
	then $\alpha^r = a_n\, a_{n-1} \cdots a_1$
Prefix, Suffix, Substring:	In $\alpha\beta\gamma$
	α is a prefix,
	γ is a suffix,
	β is a substring

Figure 1.1 - Operations on Strings

- **<u>The Chomsky Hierarchy:</u>**

- Four levels: regular, context-free, context-sensitive, and recursively enumerable languages
- Each level corresponds to a class of grammars and an equivalent class of automata
- The hierarchy is strictly nested: each class is a proper subset of the next, more powerful class
- Introduced by linguist Noam Chomsky (1956) to classify the generative power of grammars

<u>Concept Diagram: The Chomsky Hierarchy:</u>

Regular Languages → Context-Free Languages → Context-Sensitive Languages → Recursively Enumerable Languages.

- This unit formally defines deterministic finite automata, the simplest model of computation, and develops skills for tracing and designing them.

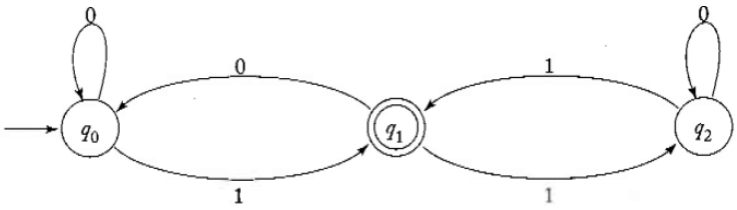
- A **finite automaton** is **a mathematical machine for determining whether a string is contained within some language.** Each finite automaton consists of a set of states connected by transitions.

A **deterministic finite accepter** or **dfa** is defined by the quintuple

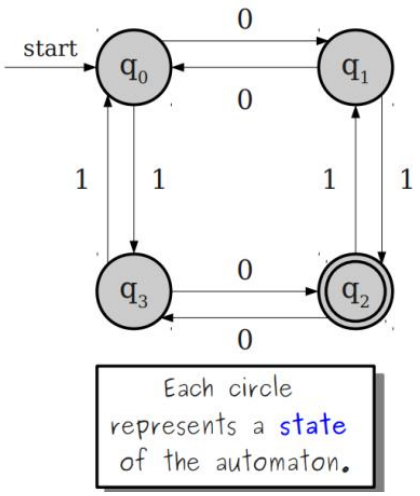
$$M = (Q, \Sigma, \delta, q_0, F),$$

where

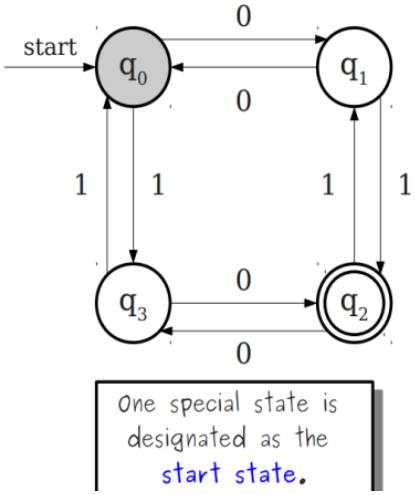
Q is a finite set of **internal states,**
Σ is a finite set of symbols called the **input alphabet,**
$\delta : Q \times \Sigma \to Q$ is a total function called the **transition function,**
$q_0 \in Q$ is the **initial state,**
$F \subseteq Q$ is a set of **final states.**



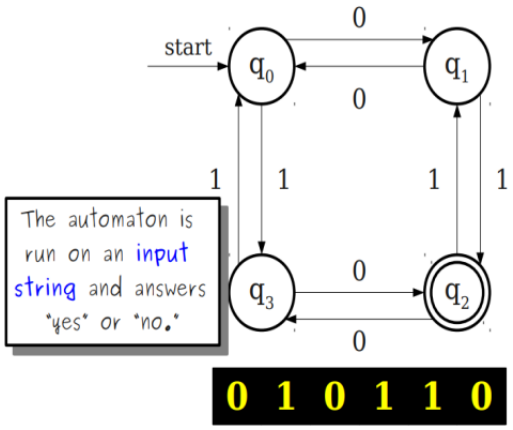
A Simple Finite Automaton



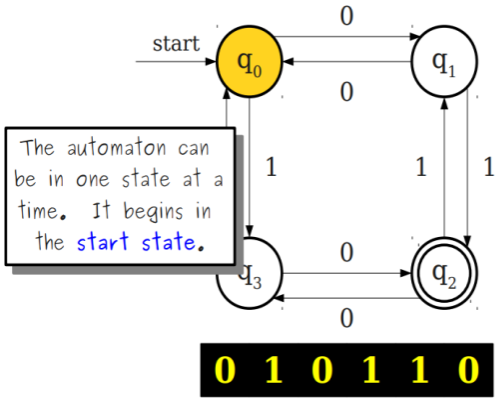
A Simple Finite Automaton



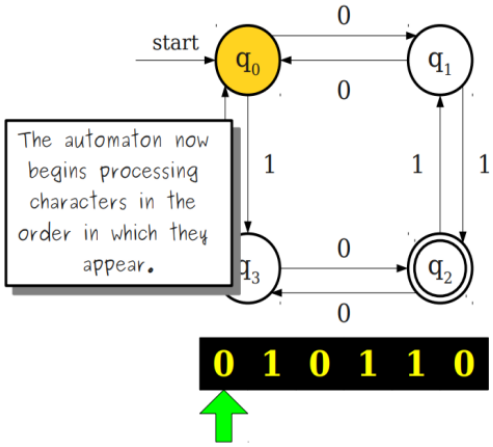
A Simple Finite Automaton



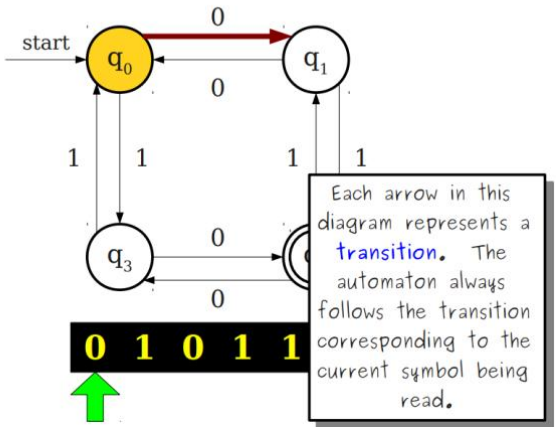
A Simple Finite Automaton



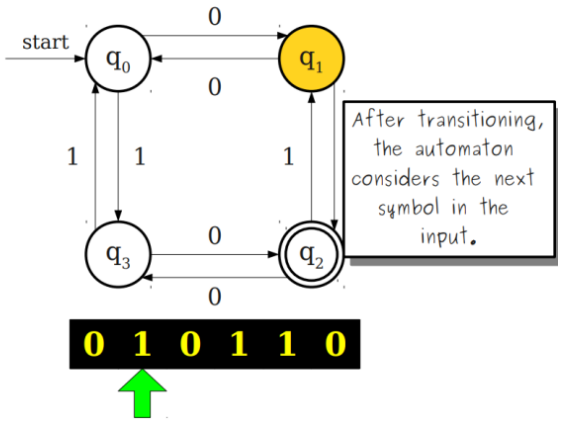
A Simple Finite Automaton



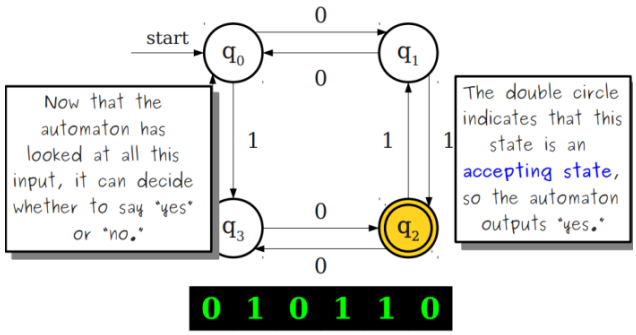
A Simple Finite Automaton



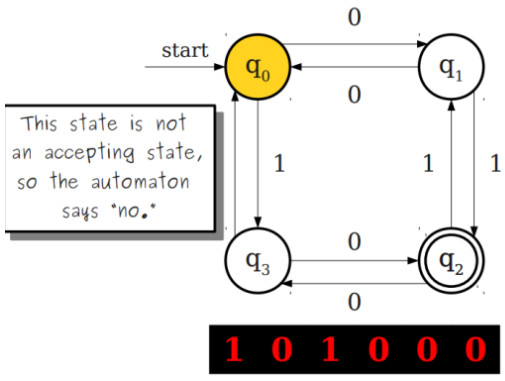
A Simple Finite Automaton



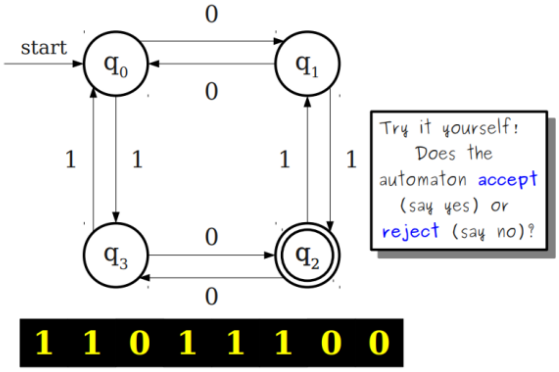
A Simple Finite Automaton



A Simple Finite Automaton

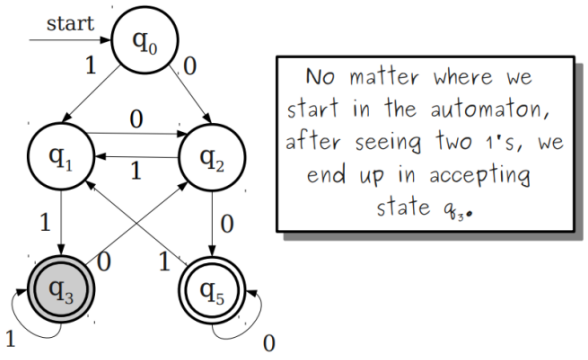


A Simple Finite Automaton

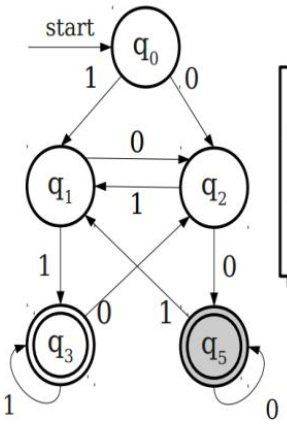


- A finite automaton is a collection of states joined by transitions.
- Some state is designated as the start state. ● Some states are designated as accepting states.
- The automaton processes a string by beginning in the start state and following the indicated transitions.
- If the automaton ends in an accepting state, it accepts the input. Otherwise, the automaton rejects the input.
- A finite automaton does not accept as soon as the input enters an accepting state.
- A finite automaton accepts if it ends in an accepting state.

What Does This Accept?

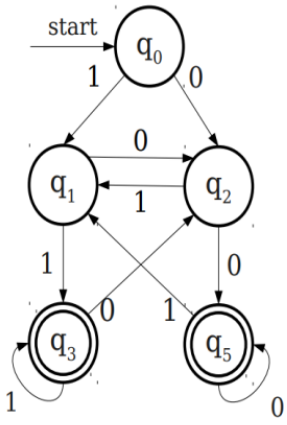


What Does This Accept?



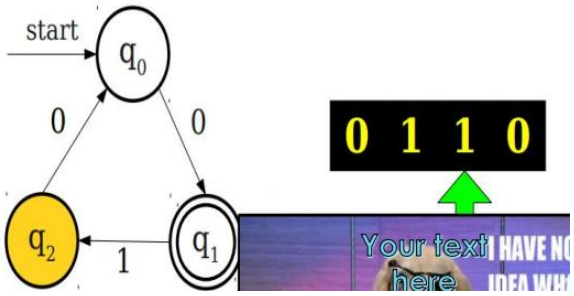
No matter where we start in the automaton, after seeing two 0's, we end up in accepting state q_5.

What Does This Accept?



This automaton accepts a string iff it ends in 00 or 11.

A Small Problem



- In order to reason about the limits of what finite automata can and cannot do, we need to specify their behavior in all cases formally.
- What happens if there is no transition out of a state on some input?
- What happens if there are multiple transitions out of a state on some input?



Input: finite string

Output: Accept or Reject

Computation process: Begin at start state, read input symbols, follow corresponding transitions,

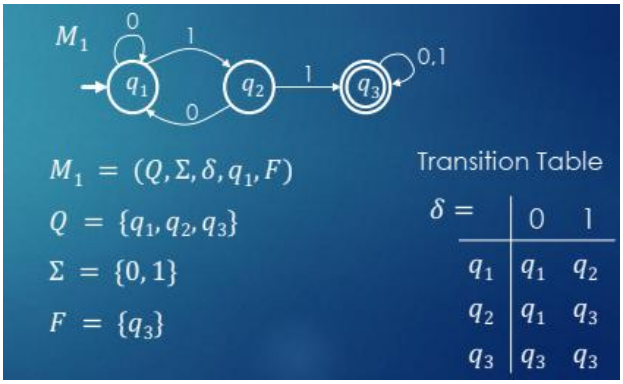
Accept if end with accept state, Reject if not.

Examples: 01101 → Accept, 00101 → Reject

M_1 accepts exactly those strings in A where $A = \{w \mid w$ contains substring 11$\}$

Say that $\boldsymbol{A}$ **is the language of** $\boldsymbol{M_1}$ and that $\boldsymbol{M_1}$ **recognizes** $\boldsymbol{A}$ and that $\boldsymbol{A = L(M_1)}$**.**

Example (Transition Table):



Define: M accepts string $w = w_1 w_2 \dots w_n$ each $w_i \epsilon \Sigma$ if there is a sequence of states r_0, $r1$, r_2, , … , $r_n \epsilon Q$

where:

- $r_0 = q_0$

- $r_i = \delta(r_{i-1}, w_i)$ for $1 \leq i \leq n$

- $r_n \epsilon F$

Recognizing languages

- $L(M) = \{w | M \text{ accepts } w\}$

- $L(M)$ is **the language of** M

- M **recognizes** $L(M)$

Define: A language is **regular** if some finite automaton recognizes it.